Universidade do Estado de Minas Gerais - UEMG

Curso de Sistemas de Informação

Disciplina: Banco de Dados 1

Documentação Técnica e Arquitetura
Sistema de Gestão e Agendamento de Recursos Universitários

Integrantes:
Erick Amorim Fernandes
Yuri Amorim Fernandes
Paulo Henrique Ribeiro Silva
Raul Arruda Soares
Bernardo Alves de Oliveira

9 de novembro de 2026

Sumário

1 Visão Geral da Arquitetura **2**

2 Stack Tecnológico e Infraestrutura **2**

3 Segurança, Autenticação e Qualidade **2**

4 Estrutura de Diretórios **3**

5 Modelagem de Dados e Integração (Normalização) **3**

1 Visão Geral da Arquitetura

O projeto adota o padrão arquitetural de Monolito Modular, projetado para ser altamente escalável e atender a qualquer unidade da UEMG distribuída pelo país. A aplicação foi expandida para suportar o agendamento de múltiplos tipos de espaços e recursos físicos (tais como salas de aula, auditórios, quadras poliesportivas, laboratórios e equipamentos). O sistema expõe uma API RESTful (Application Programming Interface), separando as responsabilidades lógicas e centralizando as regras de negócio no *backend*.

2 Stack Tecnológico e Infraestrutura

A escolha das tecnologias baseou-se em critérios de eficiência, modernidade, padronização de mercado e capacidade de evolução contínua:

- **Linguagem e Framework:** Python 3.x com Flask, provendo flexibilidade estrutural através de *Blueprints*.
- **Mapeamento Objeto-Relacional (ORM):** Utilização do **SQLAlchemy** para abstrair as interações com o banco de dados, permitindo a manipulação das tabelas e registros diretamente como objetos Python, garantindo maior segurança contra *SQL Injection* e facilitando a manutenção.
- **Versionamento de Banco de Dados:** Adoção do **Alembic** (via Flask-Migrate) para a criação e gestão das *migrations*. Isso permite o controle de versão estrutural do banco de dados, facilitando atualizações, adições de novas tabelas ou colunas, e eventuais *rollbacks* de forma segura em qualquer ambiente.
- **Banco de Dados:** PostgreSQL, escolhido por sua robustez e capacidade avançada no controle de concorrência e integridade referencial.
- **Infraestrutura:** Docker e Docker Compose para conteinerização isolada do *backend* e do banco de dados, assegurando replicabilidade.

3 Segurança, Autenticação e Qualidade

Para proteger o acesso e garantir a integridade do ecossistema educacional e da comunidade:

- **JSON Web Tokens (JWT):** Autenticação *stateless* utilizando a biblioteca `Flask-JWT-Exten`
- **RBAC (Role-Based Access Control):** Controle de acesso refinado implementado através de uma arquitetura normalizada de perfis. As permissões não são

mais valores fixos, mas gerenciadas dinamicamente na base de dados, permitindo que um mesmo usuário possua múltiplos papéis (ex: Aluno e Membro de Projeto), controlando de forma estrita o acesso a recursos críticos.

- **Proteção de Credenciais:** Senhas mantidas estritamente em formato de *hash*.
- **Observabilidade e Documentação:** Trilhas de auditoria via `Loguru` e contratos de API expostos via Swagger/OpenAPI.

4 Estrutura de Diretórios

A estrutura foi ampliada para contemplar os novos requisitos de versionamento de banco e a separação de instâncias e domínios:

```
/agendamento-recursos-uemg
 /alembic (ou /migrations) # Versionamento do banco de dados (Migrations)
 /app
    /models        # Modelagem ORM (SQLAlchemy) em 3FN
    /routes        # Controladores / Endpoints REST
    /services      # Regras de Negócio e Lógica de Agendamento
    /schemas       # Serialização e Validação de Inputs
    /utils         # Utilitários, Logging e Formatadores
 docker-compose.yml
 Dockerfile
 requirements.txt
```

5 Modelagem de Dados e Integração (Normalização)

A modelagem do banco de dados foi completamente refatorada para atingir a Terceira Forma Normal (3FN). Valores pré-definidos (*enums*) que engessavam o sistema foram extraídos para **Tabelas de Domínio/Referência**, garantindo escalabilidade interna para futuras atualizações sem a necessidade de expô-las em endpoints públicos. Entre as melhorias destacam-se:

- **Tabelas Dimensionais:** Criação das entidades `UNIDADE`, `CATEGORIA_RECURSO` e `STATUS_AGENDAMENTO`, permitindo que o sistema gerencie filiais da UEMG e diferentes tipos de laboratórios ou quadras sem alterar o código.
- **Tabelas Intermediárias (M:N):** Resolução das cardinalidades de muitos-para-muitos através das tabelas `USUARIO_PERFIL` (permitindo a um usuário ter múltiplos papéis no sistema) e `PARTICIPANTE_AGENDAMENTO` (permitindo vincular vários usuários a um mesmo agendamento de espaço físico).

O Diagrama Entidade-Relacionamento (DER) refletindo essa nova arquitetura expansível é apresentado na Figura 1.

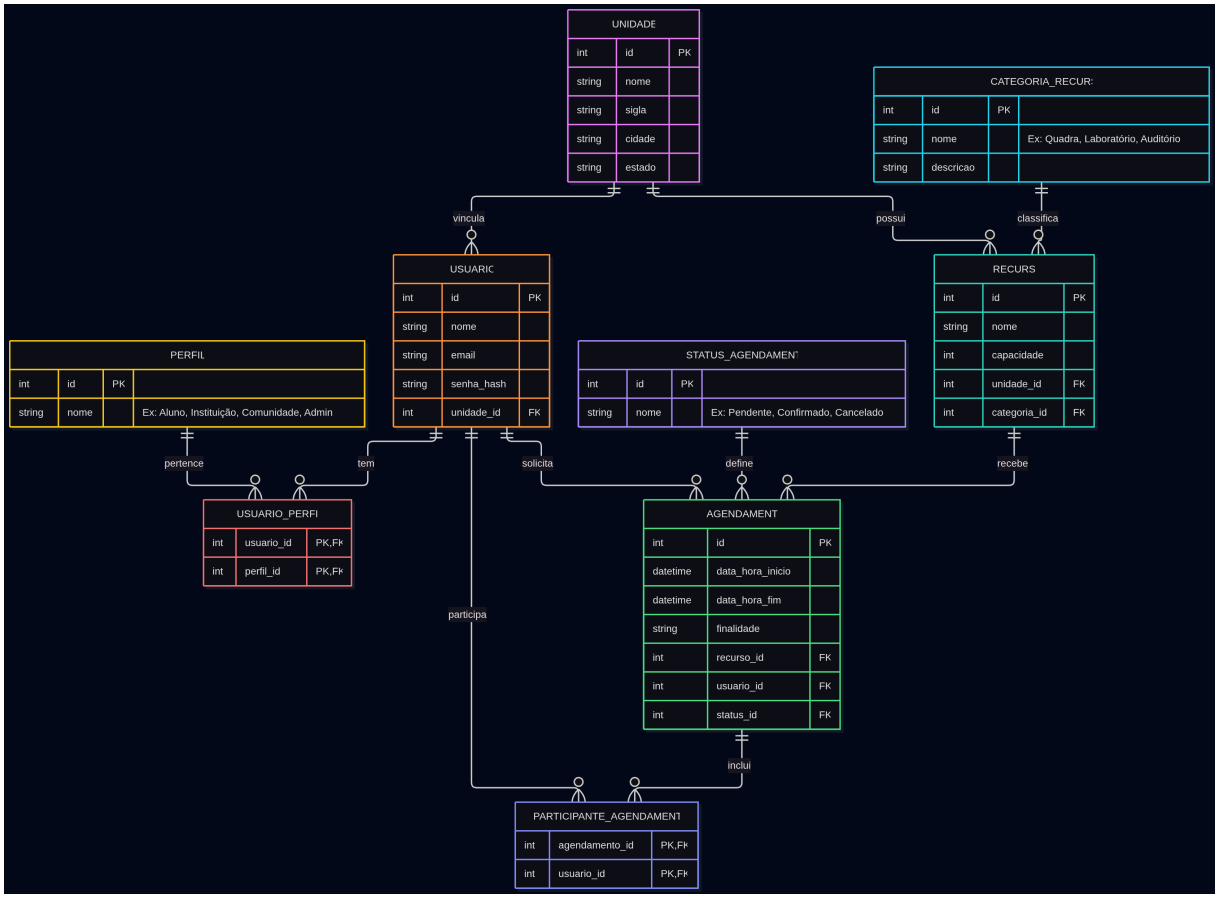


Figura 1: Diagrama Entidade-Relacionamento (DER) normalizado para múltiplas unidades e tipos de recursos.